

fitgen

2026-09-23

Table of contents

Packages and data	1
Packages	1
Example dataset	2
Filtering and normalization	2
Construction of the reduced dataset	3
Model fitting	4
Fitting a model to a single row	4
Poisson model	5
Negative binomial model	7
Binomial model: logistic regression	10
Fitting one model per row	11
Poisson models	11
Logistic regression	15
Negative binomial models	17
Fitting alternative models	21

Packages and data

The following packages are required to run the examples in this vignette.

Packages

```
pacman::p_load(fitgen, SummarizedExperiment, ggplot2, edgeR, TCGAbiolinks)
```

Example dataset

The functions in `fitgen` require a response matrix containing feature-level counts and an associated model matrix containing the sample-level covariates.

RNA-seq data are commonly stored in a `RangedSummarizedExperiment` object, provided by the `SummarizedExperiment` package. In this vignette, we use a reduced version of the TCGA COAD dataset (TCGA-COAD). The complete dataset contains gene-level expression counts together with clinical and molecular information for tumour and normal breast tissue samples.

The code used to download and prepare the original TCGA-COAD dataset is shown below for reproducibility. Because downloading and processing the complete dataset is computationally intensive, this code is not evaluated when the vignette is rendered.

```
options(timeout = 600)
query <- GDCquery(
  project = "TCGA-COAD",
  data.category = "Transcriptome Profiling",
  data.type = "Gene Expression Quantification",
  workflow.type = "STAR - Counts"
)
GDCdownload(
  query,
  method = "api",
  files.per.chunk = 30,
  directory = "GDCdata"
)
y <- GDCprepare(query, directory = "GDCdata") ##
save(y, file = "tcga_coad.rda")
```

Filtering and normalization

Lowly expressed features are removed before model fitting. Features with duplicated gene symbols are also excluded so that each retained symbol is represented only once.

Differences in library composition and library size are accounted for by estimating normalization factors with the TMM method implemented in `edgeR`. These factors are stored in the `DGEList` object; the original count matrix is retained. Gene identifiers are also processed to remove the version suffix from the Ensembl identifiers.

The resulting object is saved for use in the subsequent preparation of the reduced example dataset.

```
load("tcga_coad.rda")
sel1 = which(colData(y)$sex_at_birth != "unknown")
sel2 = which(!is.element(colData(y)$sample_type,
                        c("Metastatic", "Recurrent Tumor")))
y = y[,intersect(sel1, sel2)]
colData(y)$sex_at_birth = factor(colData(y)$sex_at_birth)
colData(y)$sample_type = factor(colData(y)$sample_type)
```

Construction of the reduced dataset

To keep the examples computationally manageable, a reduced dataset is constructed from the complete TCGA-COAD data.

Only the clinical variables used in the examples are retained. Samples with missing values for these variables are excluded, as are metastatic and recurrent tumour samples. Lowly expressed features are then filtered out.

For the construction of this reduced dataset, adjusted count values are obtained from an **edgeR** object after estimating a common dispersion parameter. These values are rounded and stored in the assay of the reduced object. Finally, a random subset of 100 features is selected using a fixed random seed so that the example is reproducible.

This preprocessing step is shown for documentation purposes and is not evaluated when the vignette is rendered.

```
vars = c("sex_at_birth", "sample_type")
colData(y) = colData(y)[,vars]
x0 = colData(y)[,vars]
selcol = which(complete.cases(x0))
to_keep = rowSums(cpm(assay(y)) > 0.5) > (ncol(y)/100)
table(to_keep)
y = y[to_keep,]
dge = edgeR::DGEList(counts=assay(y))
dge.c = edgeR::estimateCommonDisp(dge)
dge.c$common.dispersion ## 1.041049
counts0 = dge.c$pseudo.counts
rownames(counts0) = rownames(assay(y))
colnames(counts0) = colnames(assay(y))
assay(y) = round(counts0)
set.seed(1234)
```

```
y = y[sample(nrow(y),100),]
save(y,file="y.rda")
```

The resulting reduced object, `y`, is included in the `fitgen` package and can be loaded directly as follows.

```
data(y)
```

The rows of `y` correspond to features and the columns correspond to samples. The assay contains the response values used in the examples, whereas the sample-level variables required to construct the model matrix are stored in `colData(y)`.

Model fitting

Fitting a model to a single row

We first illustrate the main fitting functions using a single row of the response matrix. The same sample-level covariates are used in all models.

The variables `sex_at_birth` and `sample_type` are included as common predictors. The corresponding model matrix, `xcommon`, is constructed from the sample metadata stored in `colData(y)`.

```
vars = c("sex_at_birth","sample_type")
ff = paste0("~ ",paste(vars,collapse="+"))
ff = as.formula(ff)
xcommon = model.matrix(ff,data=colData(y))
summary(as.data.frame(colData(y))[,vars])
```

sex_at_birth	sample_type
female:248	Primary Tumor :479
male :272	Solid Tissue Normal: 41

We select a row from the assay matrix. The resulting vector, `y0`, contains the response values for this feature across all samples.

A binary response, `y1`, is also constructed to illustrate logistic regression. Observations above the 60th percentile of `y0` are assigned the value 1, and the remaining observations are assigned the value 0. This dichotomization is used only for illustration and is not intended as a general preprocessing strategy for RNA-seq data.

```
i = 50
yt = assay(y)
y0 = yt[i,]
quantile(y0,probs=.6)
```

```
60%
66
```

```
y1 = (y0 > quantile(y0,probs=.6))* 1. ## For logistic regression
x0 = colData(y)[,vars]
df0 = data.frame(x0,y0,y1)
names(df0)
```

```
[1] "sex_at_birth" "sample_type" "y0"          "y1"
```

Poisson model

We first fit a Poisson generalized linear model using `stats::glm()`. This fit serves as a reference for comparison with the corresponding functions implemented in `fitgen`.

The model uses a log link and assumes that the conditional variance is equal to the conditional mean:

$$\text{Var}(Y_j | \mathbf{x}_j) = \mu_j.$$

The coefficients, deviance, and fitted values obtained from this reference model are displayed below.

```
fit0 = glm(y0~ sex_at_birth + sample_type,family = "poisson",data=df0)
coef(fit0)
```

	(Intercept)	sex_at_birthmale
	4.22034739	-0.04626339
sample_typeSolid Tissue Normal		
	-1.03432313	

```
deviance(fit0)
```

```
[1] 13684.32
```

```
head(fitted(fit0))
```

```
TCGA-AA-3561-01A-01R-0821-07 TCGA-AZ-4616-01A-21R-1839-07
                        64.98029                        68.05712
TCGA-AA-3712-11A-01R-1723-07 TCGA-AY-A71X-01A-12R-A37K-07
                        23.09834                        68.05712
TCGA-AA-3861-01A-01R-1022-07 TCGA-AZ-6603-11A-02R-1839-07
                        64.98029                        24.19205
```

We next fit the same Poisson model using `fitgen::StGLM()`. The response vector, model matrix, distribution, and offset are supplied explicitly. Setting `retmu = TRUE` indicates that the fitted means should be returned.

The resulting object contains the model deviance, the estimated regression coefficients, and the fitted means. Because the same response, model matrix, link function, and variance specification are used, the results can be compared directly with those obtained from `stats::glm()`.

```
Fit0 = StGLM(y=y0,X=xcommon,model="poisson",offset=rep(0,length(y0)),
             warn_me=TRUE,retmu=TRUE)
Fit0$Dev ## deviance
```

```
[1] 13684.32
```

```
Fit0$beta ## coefficients
```

```
[1] 4.22034739 -0.04626339 -1.03432313
```

```
head(Fit0$muest) ## fitted values (estimates for the observations and means)
```

```
[1] 64.98029 68.05712 23.09834 68.05712 64.98029 24.19205
```

The function `fitgen::glmRow()` provides an interface similar to that of `stats::glm()`. Standard extractor functions such as `coef()`, `coefficients()`, `deviance()`, and `fitted()` can therefore be used with the fitted object.

Setting `is_fitted_values = TRUE` indicates that the fitted values should be stored and returned.

```
fit = glmRow(x=xcommon,y=y0,family="poisson",is_fitted_values=TRUE)
coef(fit)
```

```
[1] 4.22034739 -0.04626339 -1.03432313
```

```
coefficients(fit)
```

```
[1] 4.22034739 -0.04626339 -1.03432313
```

```
deviance(fit)
```

```
[1] 13684.32
```

```
head(fitted(fit))
```

```
[1] 64.98029 68.05712 23.09834 68.05712 64.98029 24.19205
```

The three procedures fit the same Poisson model to the selected row. Their estimated coefficients, deviances, and fitted values should therefore agree, up to minor numerical differences arising from the fitting algorithms.

Negative binomial model

We next fit an NB2 regression model to the same response vector. Under the NB2 parameterization used here, the conditional variance is

$$\text{Var}(Y_j | \mathbf{x}_j) = \mu_j + \phi \mu_j^2,$$

where ϕ is the negative binomial dispersion parameter.

We first fit the model using `stats::glm()` with the negative binomial family provided by the `MASS` package. This fit serves as a reference for comparison with the corresponding functions implemented in `fitgen`.

The `MASS::negative.binomial()` family expresses the dispersion in terms of `theta`. Under the parameterization used here,

$$\theta = \frac{1}{\phi}.$$

In this example, ϕ is fixed at 0.7983342, so the corresponding value `theta = 1 / 0.7983342` is supplied to `MASS::negative.binomial()`.

The estimated coefficients, deviance, and fitted values are displayed below.

```
fit0 = glm(y0~ sex_at_birth + sample_type,
           family = MASS::negative.binomial(theta=1/0.7983342),data=df0)
coef(fit0)
```

```

              (Intercept)              sex_at_birthmale
              4.21096668              -0.02850242
sample_typeSolid Tissue Normal
              -1.03107527
```

```
deviance(fit0)
```

```
[1] 313.7847
```

```
head(fitted(fit0))
```

```

TCGA-AA-3561-01A-01R-0821-07 TCGA-AZ-4616-01A-21R-1839-07
              65.52713              67.42168
TCGA-AA-3712-11A-01R-1723-07 TCGA-AY-A71X-01A-12R-A37K-07
              23.36850              67.42168
TCGA-AA-3861-01A-01R-1022-07 TCGA-AZ-6603-11A-02R-1839-07
              65.52713              24.04414
```

We next fit the same NB2 model using `fitgen::StGLM()`. The response vector, model matrix, distribution, offset, and dispersion parameter are supplied explicitly.

The argument `Phi` represents the NB2 dispersion parameter ϕ . In this fit, ϕ is treated as fixed while the regression coefficients are estimated. Setting `retmu = TRUE` indicates that the fitted means should be returned.

The resulting object contains the model deviance, the estimated regression coefficients, and the fitted means. Because the same response, model matrix, link function, and fixed value of ϕ are used, the results can be compared directly with those obtained from `stats::glm()`.

```
Fit0 = StGLM(y=y0,X=xcommon,model="negbin",offset=rep(0,length(y0)),
            warn_me=TRUE,retmu=TRUE,Phi=0.7983342)
Fit0$Dev ## deviance
```

```
[1] 313.7847
```

```
Fit0$beta ## coefficients
```

```
[1] 4.21096730 -0.02850328 -1.03107749
```

```
head(Fit0$muest) ## fitted values (estimates for the observations and means)
```

```
[1] 65.52711 67.42173 23.36844 67.42173 65.52711 24.04410
```

The function `fitgen::NB2Row()` provides a single-row interface similar to that of `stats::glm()`. Standard extractor functions such as `coef()`, `coefficients()`, `deviance()`, and `fitted()` can therefore be used with the fitted object.

The argument `phi` represents the fixed NB2 dispersion parameter ϕ .

Setting `is_fitted_values = TRUE` indicates that the fitted values should be stored and returned.

```
fit = NB2Row(x=xcommon,y=y0,phi=0.7983342,is_fitted_values=TRUE)
coef(fit)
```

```
[1] 4.21096730 -0.02850328 -1.03107749
```

```
coefficients(fit)
```

```
[1] 4.21096730 -0.02850328 -1.03107749
```

```
deviance(fit)
```

```
[1] 313.7847
```

```
head(fitted(fit))
```

```
[1] 65.52711 67.42173 23.36844 67.42173 65.52711 24.04410
```

The three procedures fit the same NB2 model to the selected row using the same fixed value of ϕ . Their estimated coefficients, deviances, and fitted values should therefore agree, up to minor numerical differences arising from the fitting algorithms.

Binomial model: logistic regression

Finally, we illustrate the use of `fitgen::glmRow()` with a binary response. The variable `y1` indicates whether each observation is above the 60th percentile of the response values for the selected feature.

We first fit a logistic regression model using `stats::glm()`. We then fit the same model using `fitgen::glmRow()`. Both fits use a binomial response with a logit link and the same model matrix.

The estimated coefficients and deviance are extracted from both fitted objects. For the model fitted with `glmRow()`, setting `is_fitted_values = TRUE` indicates that the fitted probabilities should be stored and returned.

```
fit1 = glm(y1~ sex_at_birth + sample_type,family = "binomial",data=df0)
coef(fit1)
```

```
              (Intercept)              sex_at_birthmale
              -0.23901690              -0.09745534
sample_typeSolid Tissue Normal
              -17.27978847
```

```
deviance(fit1)
```

```
[1] 653.7832
```

```
fit2 = glmRow(x=xcommon,y=y1,family="binomial",is_fitted_values=TRUE)
coefficients(fit2)
```

```
[1] -0.23901690 -0.09745534 -28.27978851
```

```
coef(fit2)
```

```
[1] -0.23901690 -0.09745534 -28.27978851
```

```
deviance(fit2)
```

```
[1] 653.7832
```

```
head(fitted(fit2))
```

```
[1] 4.166667e-01 4.405286e-01 3.733498e-13 4.405286e-01 4.166667e-01  
[6] 4.115667e-13
```

The two procedures fit the same logistic regression model to the selected row. Their estimated coefficients and deviances should therefore agree, up to minor numerical differences arising from the fitting algorithms. The fitted values returned by `glmRow()` are the estimated probabilities of observing a value equal to 1.

This example is included only to illustrate the use of the row-wise interface with a binomial response. The dichotomization of the count data should not be interpreted as a recommended procedure for differential expression analysis, as it discards information from the original response.

Fitting one model per row

Poisson models

We now fit a separate generalized linear model to each row of the response matrix `yt`. All row-wise models include the same sample-level covariates, represented by the common model matrix `xcommon`.

Two model specifications are considered. The first includes only the predictors in `xcommon`. The second extends `xcommon` with row-specific predictors derived from neighboring features. For each target row, the expression profiles of the neighbors identified by `fitgen::find_neighbours()` are added as additional columns to `xcommon`. The resulting extended model matrix is therefore specific to each target row.

The model containing only `xcommon` describes the association between the sample-level covariates and the response for each feature without conditioning on neighboring features. By contrast, the neighborhood-adjusted model describes this association conditional on the expression profiles of the selected neighbors.

Comparing the two specifications makes it possible to assess whether conditioning on locally related features changes the fitted means, residual variability, or estimated effects of the sample-level covariates. The two specifications should therefore be interpreted as related but distinct models.

In this example, five neighboring features are used to extend `xcommon` for each row-wise model. Because identifying the neighbors can be computationally intensive, this step is not evaluated when the vignette is rendered. The resulting object is saved and reused in the subsequent examples.

```
neighbours = find_neighbours(yt,neighbours_n=5) ## Slow processing
```

Cargando paquete requerido: memuse

The neighbors identified for row 50 are shown below.

```
neighbours[i,]
```

```
[1] 78 14 19 99 20
```

We now fit a separate Poisson generalized linear model to each row of `yt` using `fitgen::glmRows()`.

All row-wise models include the predictors in `xcommon`. For each target row, the expression profiles of its previously identified neighbors are added as additional columns to the common model matrix. Thus, each full model contains a common component, shared by all rows, and a row-specific component defined by the selected neighbors.

The argument `selected` is a logical vector indicating the rows for which a model should be fitted. In this example, all rows are selected. Setting `is_fitted_values = TRUE` indicates that the fitted values should be stored and returned.

Because the neighbors have already been identified, they are supplied directly through the `neighbours` argument. Fitting one model per row with additional neighbor-based predictors can be computationally intensive. Therefore, this step is not evaluated when the vignette is rendered, and the previously fitted object is saved for use in the following sections.

```
fits = glmRows(x=xcommon,y=yt,family="poisson",
               is_normalized=TRUE,
               selected = rep(TRUE,nrow(yt)),
               is_fitted_values=TRUE, ## The fitted values are returned
               neighbours = neighbours)
```

The object `fits` contains the row-wise Poisson fits obtained using the common predictors and the additional neighbor-based predictors. This object belongs to the `fitgen_Lglm` class.

```
class(fits)
```

```
[1] "fitgen_Lglm"
```

S3 methods are available for objects of class `fitgen_Lglm`. These methods extend the usual extractor functions for generalized linear models to an object containing multiple row-wise fits.

For each row, results can be extracted from two model specifications. With `type = "common"`, the results correspond to the model containing only the predictors in `xcommon`. With `type = "full"`, the results correspond to the extended model containing both the common predictors and the row-specific neighbor-based predictors.

The following code extracts the coefficients, fitted values, and deviances from both model specifications. It also computes a feature-wise quasi-dispersion estimate using the Wedderburn method.

```
a = coef(fits,type="common")
a = coefficients(fits,type="common") ## Only common predictors are used
a[[1]] ## the first vector of coefficients
```

```
[1] 2.2446350 -0.2571915 0.7857462
```

```
a = coef(fits,type="full")
a = coefficients(fits,type="full") ## Common and neighbor-based predictors are used
a[[1]] ## the first vector of coefficients
```

```
[1] 1.797924203 -0.214270346 1.144883684 0.008690771 0.017664805
[6] 0.006803938 0.008189302 0.001796905
```

```
b = fitted(fits,type="common") ## Fitted values with common predictors
head(b[[1]])
```

```
[1] 7.296856 9.436971 16.009619 9.436971 7.296856 20.705125
```

```
b = fitted(fits,type="full") ## ## Fitted values with common and neighbor-based predictors
head(b[[1]])
```

```
[1] 5.341797 6.335547 17.581932 7.801374 4.971026 20.909333
```

```
c = deviance(fits,type="common") ## Deviances with common predictors
c[[1]] ## the first one
```

```
[1] 8030.692
```

```
summary(unlist(c))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1517	3778	36434	552529	117005	46937337

```
c = deviance(fits,type="full") ## Deviances with all predictors  
c[[1]] ## the first one
```

```
[1] 2960.611
```

```
summary(unlist(c))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
599	2114	20040	508064	87754	44731541

```
## Quasi dispersion with common predictors where type can be  
## "Wedderburn", "Farrington" or "Fletcher"  
d = quasidisersion(fits,type="Wedderburn")  
d[[1]] ## the first one
```

```
[1] 6.759683
```

```
summary(unlist(d))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
2.072e+00	6.567e+00	4.422e+01	1.591e+03	1.897e+02	1.448e+05

We can also fit the row-wise Poisson models without neighbor-based predictors. In this case, each model contains only the predictors in the common model matrix `xcommon`.

This specification provides the unadjusted model for each feature and can be used as a reference when assessing the effect of including neighboring features. Because no neighbor-based predictors are included, the common and full model specifications coincide.

As before, all rows are selected and the fitted values are requested. This computationally intensive step is not evaluated when the vignette is rendered, and the resulting object is saved for subsequent use.

```
fits = glmRows(x=xcommon,y=yt,family="poisson",
               is_normalized=TRUE,
               selected = rep(TRUE,nrow(yt)),
               is_fitted_values=TRUE,
               neighbours=NULL,
               neighbours_n = 0)
```

Logistic regression

To illustrate row-wise logistic regression, the count matrix `yt` is dichotomized to obtain a binary response matrix `y1`. In this example, counts greater than or equal to 100 are assigned the value 1, whereas counts below 100 are assigned the value 0.

This transformation is introduced solely to demonstrate the use of `fitgen::glmRows()` with a binomial response. It should not be interpreted as a generally recommended transformation for RNA-seq differential expression analysis, since dichotomization discards information contained in the original count data.

```
y1 = (yt >= 100)*1
summary(apply(y1,1,sum))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.0	0.0	170.5	246.8	519.2	520.0

We now fit a separate logistic regression model to each row of the binary response matrix `y1`.

All models include the predictors in `xcommon`. For each target row, the expression profiles of its previously identified neighbors are added as additional columns to the common model matrix. Thus, each full model contains a common component and a row-specific component defined by the selected neighbors.

The previously computed neighbors are supplied through the `neighbours` argument. All rows are selected, and setting `is_fitted_values = TRUE` indicates that the fitted probabilities should be stored and returned.

Because fitting one neighborhood-adjusted model per row can be computationally intensive, this step is not evaluated when the vignette is rendered. The resulting object is saved for subsequent inspection.

```
fits = glmRows(x=xcommon,y=y1,family="binomial",
               is_normalized=TRUE,
               selected = rep(TRUE,nrow(y1)),
               is_fitted_values=TRUE, ## The fitted values are returned
               neighbours = neighbours) ## Previously calculated

save(fits,file=~/.Dropbox/fitgen-data/fits_with_neighbours_logistic_tcga_coad.rda")
```

We now load the previously fitted object containing the row-wise logistic regression models.

```
load("~/.Dropbox/fitgen-data/fits_with_neighbours_logistic_tcga_coad.rda")
```

The availability of the deviance provides a simple diagnostic of whether a model produced a valid result for each row. The following code extracts the deviances from the full models, which include both the common predictors and the row-specific neighbor-based predictors.

We then count the number of missing deviance values and summarize the distribution of the available values.

```
a = deviance(fits,type="full")
table(is.na(unlist(a)))
```

```
FALSE  TRUE
   90    10
```

```
summary(unlist(a))
```

```
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
 0.00    0.00    0.00   90.65   23.83   676.96     10
```

The remaining extractor methods can be used in the same way to obtain coefficients, fitted probabilities, and quasi-dispersion estimates from the row-wise logistic regression models.

Next, we fit the same row-wise logistic regression models **without** neighbor-based predictors. In this case, each model contains only the predictors in the common model matrix `xcommon`.

This specification provides the unadjusted model for each feature and can be used as a reference for assessing the effect of including neighboring features.

```
fits = glmRows(x=xcommon,y=y1,family="binomial",
               is_normalized=TRUE,
               selected = rep(TRUE,nrow(y1)),
               is_fitted_values=TRUE,
               neighbours = NULL, ## No neighbours are provided
               neighbours_n = 0) ## Number of neighbours is null
```

Negative binomial models

We now fit a separate NB2 regression model to each row of the response matrix `yt`.

Under the NB2 parameterization used here, the conditional variance for each feature is

$$\text{Var}(Y_j \mid \mathbf{x}_j) = \mu_j + \phi \mu_j^2,$$

where ϕ is the negative binomial dispersion parameter.

In this example, a feature-specific value of ϕ is estimated using the tagwise dispersion method implemented in `edgeR`. This approach allows the NB2 dispersion parameter to vary across features while borrowing information across genes during estimation.

The dispersion estimates are obtained before fitting the row-wise NB2 models. Because this step can be computationally intensive, it is not evaluated when the vignette is rendered. The resulting vector of feature-specific dispersion estimates is saved for use in the subsequent models.

```
phi = NB2_phi(y=yt,method="tagwise_edgeR",selected=rep(TRUE,nrow(yt)))
```

We now use the previously computed tagwise dispersion estimates to fit one NB2 model to each row of `yt`.

All models include the predictors in `xcommon`. For each target row, the expression profiles of its previously identified neighbors are added as additional columns to the common model matrix. The resulting full model therefore contains both the common predictors and the row-specific neighbor-based predictors.

The object `phi` contains the feature-specific NB2 dispersion estimates previously obtained using the tagwise method implemented in `edgeR`. These externally estimated values are supplied to `fitgen::NB2Rows()` and used during the row-wise model fitting. Because `phi` is provided explicitly, the dispersion parameters are not re-estimated at this stage.

All rows are selected, and setting `is_fitted_values = TRUE` indicates that the fitted values should be stored and returned. Because fitting one neighborhood-adjusted NB2 model per row

can be computationally intensive, this step is not evaluated when the vignette is rendered. The resulting object is saved for subsequent inspection.

```
fits = NB2Rows(x=xcommon,
               y=yt,
               phi=phi,
               phi_method = "single",
               is_normalized=TRUE,
               selected = rep(TRUE,nrow(yt)), ## All rows are selected
               is_fitted_values=TRUE, ## The fitted values are returned
               neighbours = neighbours)
```

```
Warning in fitgen::FitGLMWithNei(dat = y, normalized = is_normalized, Cl =
neighbours, : Row 58: the extended model could not be fitted even with one
neighbour.
```

```
Warning in fitgen::FitGLMWithNei(dat = y, normalized = is_normalized, Cl =
neighbours, : Row 57: the base model fitted, but not even the model with one
neighbour could fit. Data for the extended model will be NULL.
```

The object `fits` contains the row-wise NB2 fits obtained using the feature-specific tagwise dispersion estimates.

For each row, results can be extracted from two model specifications. With `type = "common"`, the results correspond to the model containing only the predictors in `xcommon`. With `type = "full"`, the results correspond to the extended model containing both the common predictors and the row-specific neighbor-based predictors.

The following code extracts the coefficients, fitted values, and deviances from both model specifications. It also computes feature-wise quasi-dispersion estimates using the Wedderburn method and NB2 dispersion estimates using the Bliss method.

The NB2 dispersion parameter and the quasi-dispersion parameter have different roles. The NB2 dispersion parameter ϕ defines the baseline quadratic mean–variance relationship, whereas the quasi-dispersion parameter measures the remaining variability relative to that variance specification after the mean model has been fitted.

```
a = coef(fits,type="common")
a = coefficients(fits,type="common") ## Only common predictors are used
a[[1]] ## the first vector of coefficients
```

```
[1] 2.2547205 -0.2830672 0.8073970
```

```
a = coef(fits,type="full")
a = coefficients(fits,type="full") ## Common and neighbor-based predictors are used
a[[1]] ## the first vector of coefficients
```

```
[1] 1.777845532 -0.057976398 1.123490251 0.006825779 0.012353823
[6] 0.015995209 0.002819202 0.001887464
```

```
b = fitted(fits,type="common") ## Fitted values with common predictors
head(b[[1]]) ##
```

```
[1] 7.182542 9.532629 16.103721 9.532629 7.182542 21.372767
```

```
b = fitted(fits,type="full") ## Fitted values with common and neighbor-based predictors
head(b[[1]])
```

```
[1] 5.983588 6.122334 18.328705 6.588157 5.636608 19.150882
```

```
c = deviance(fits,type="common") ## Deviances with common predictors
c[[1]] ## the first one
```

```
[1] 98.0363
```

```
summary(unlist(c))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
51.6	192.4	421.7	361.0	495.6	636.0

```
c = deviance(fits,type="full") ## Deviances with all predictors
c[[1]] ## the first one
```

```
[1] 72.88662
```

```
summary(unlist(c))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
0.0	184.0	311.8	297.5	418.9	579.1	3

```
d = quasidisersion(fits,type="Wedderburn") ## Feature-wise quasi-dispersion estimates
d[[1]] ## the first one
```

```
[1] 0.1003599
```

```
summary(unlist(d))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
0.0283	0.4110	0.7293	0.6324	0.8381	1.2074	3

```
e = dispersion(fits,type="Bliss") ## NB2 dispersion estimates using the Bliss method
e[[1]] ## ## The first dispersion estimate
```

```
[1] 9.34588
```

```
summary(unlist(e))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.05845	0.27790	0.60739	7.46560	6.97916	68.96051

We next fit the same row-wise NB2 models without neighbor-based predictors. In this case, each model contains only the predictors in the common model matrix `xcommon`.

The same feature-specific values of the NB2 dispersion parameter ϕ are used as in the neighborhood-adjusted models. Therefore, comparing the two specifications makes it possible to assess the effect of adding neighbor-based predictors while keeping the NB2 dispersion parameter fixed.

As before, all rows are selected and the fitted values are requested. This computationally intensive step is not evaluated when the vignette is rendered, and the resulting object is saved for subsequent use.

```
fits = NB2Rows(x = xcommon,
               y = yt,
               phi=phi,
               phi_method = "single",
               is_normalized=TRUE,
               selected = rep(TRUE,nrow(yt)),
               is_fitted_values=TRUE,
               neighbours = NULL,
               neighbours_n = 0)
```

We now load the previously fitted object containing the row-wise NB2 models without neighbor-based predictors.

Fitting alternative models

The function `fitgen::FitRows()` provides a high-level interface for fitting alternative row-wise models within a common analytical framework.

All models use the predictors in `xcommon`. For each target row, the expression profiles of the previously identified neighbors are added as row-specific predictors. The externally estimated NB2 dispersion parameters contained in `phi` are supplied to the fitting function and are not re-estimated at this stage.

In addition to the Poisson and NB2 fits, `FitRows()` evaluates quasi-dispersion using the method specified by `quasi_method`. In this example, the Wedderburn estimator is used.

The arguments `alpha` and `p.adjust.method` specify the significance level and the method used to adjust for multiple comparisons, respectively. Setting `p.adjust.method = "BH"` applies the Benjamini–Hochberg adjustment.

Because fitting and evaluating alternative models for every row can be computationally intensive, this step is not evaluated when the vignette is rendered. The resulting object is saved for subsequent inspection.

```
res = FitRows(x=xcommon,y=yt,phi = phi,
              is_normalized=TRUE,
              is_fitted_values=TRUE,
              neighbours_method="trivial",
              neighbours = neighbours)
```

```
Warning in fitgen::FitGLMWithNei(dat = y, normalized = is_normalized, C1 =
neighbours, : Row 58: the extended model could not be fitted even with one
neighbour.
```

```
Warning in fitgen::FitGLMWithNei(dat = y, normalized = is_normalized, C1 =
neighbours, : Row 57: the base model fitted, but not even the model with one
neighbour could fit. Data for the extended model will be NULL.
```

We now load the previously computed results. The returned object contains the row-wise Poisson and NB2 fits, together with the corresponding diagnostic and inferential summaries.

The class of the Poisson component is inspected below. We also summarize the model classifications stored in `res$type.model` and the feature-wise quasi-dispersion estimates obtained from the Poisson and NB2 fits.

```
class(res$Poi)
```

```
[1] "fitgen_Lglm"
```

```
summary(unlist(quasidispersion(res$Poi)))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
2.072e+00	6.567e+00	4.422e+01	1.591e+03	1.897e+02	1.448e+05

```
summary(unlist(quasidispersion(res$NB2)))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
0.0283	0.4110	0.7293	0.6324	0.8381	1.2074	3

The Poisson and NB2 quasi-dispersion estimates quantify residual variability relative to different baseline variance functions. For the Poisson fits, the baseline variance is the conditional mean. For the NB2 fits, it is determined by the conditional mean and the supplied NB2 dispersion parameter.

Therefore, quasi-dispersion estimates from the Poisson and NB2 fits should be interpreted relative to their respective baseline variance functions. Their values are not directly comparable without taking these different variance specifications into account.

The results returned by `FitRows()` support the comparison of feature-wise conclusions under alternative variance and dispersion assumptions. They should not be interpreted as identifying a universally optimal model for every feature.